



Blueprint, Software Engineering & Computing Technology @ Mo E

Bahir Dar Poly Technic College (Bahir Dar University)



Scan to open on Studocu



Test Blueprint Manual for National Exit Examination

BAND: 1

PROGRAM:
BACHELOR OF SCIENCE DEGREE IN SOFTWARE
ENGINEERING AND COMPUTING TECHNOLOGY (Artificial
Intelligence, Cyber Security, Information
Technology, Software Engineering Streams)

Compiled By:

1. Sileshi Demesie (Ph.D), Addis Ababa Institute of Technology, AAU
2. Beakal Gizachew (Ph.D), Addis Ababa Institute of Technology, AAU

February 2024

Ministry of Education
Addis Ababa, Ethiopia

Table of Contents

1. INTRODUCTION	3
2. THE OBJECTIVE OF THE TEST BLUEPRINT	4
3. STEPS FOR THE DEVELOPMENT OF THE TEST BLUEPRINT	4
4. COMPONENTS CONSIDERED IN THE TEST BLUEPRINT PREPARATION	5
4.1. PROGRAM STRUCTURE	5
4.2. GRADUATE PROFILES.....	5
4.3. OBJECTIVES OF THE PROGRAM.....	6
4.4. LEARNING OUTCOMES	8
5. THEMES AND LIST OF COURSES	11
5.1. COMMON CORE COURSES THEME, COURSES, AND LEARNING OUTCOMES	12
6. CONCLUSION	16
APPENDIX A:.....	17

1. Introduction

The Ministry of Education in Ethiopia has declared the introduction of exit examinations for all undergraduate students enrolled in both public and private institutions starting from the academic year 2023/2024. This initiative is geared towards enhancing the caliber of graduates produced by higher education establishments. The exit exams aim to assess whether students have acquired the necessary knowledge, skills, and attitudes outlined by the competency areas prepared for their respective programs. To implement this, it's necessary to prepare a plan for constructing achievement tests tailored to the competency areas identified for each specific program.

An achievement test demands very systematic and careful planning, as a fact that good planning is a symbol of success. These tests play a crucial role in evaluating student learning and program effectiveness. Tests should be able to measure student performance in all dimensions of knowledge, skill, and attitude. To achieve the valid and reliable measurement of student learning and program learning outcomes, the development of valid and reliable tests is mandatory. The carefully planned test construction contributes to improving the overall quality of the test in terms of test content validity, difficulty level, discrimination power, and test reliability.

Test preparation is not an easy task; it requires careful planning and guidelines to make the task simple. Test construction needs the preparation of a test blueprint. A test blueprint is defined as a complete plan that explains how to develop a test. The term refers to a map or specification of assessment to ensure that the assessment programs cover all aspects of the curriculum and educational domains over a specified period. It helps curriculum developers/test constructors to match various competencies with the course content and the appropriate modality of assessment.

Generally, a test blueprint will help to ensure tests: 1) Appropriately assess the achievement of instructional objectives of the course; 2) Appropriately reflect key course goals, objectives and the material learned or covered during the instruction period; and 3) Include the appropriate item formats along with the knowledge and skills being assessed.

With these objectives in mind, the team has prepared a test blueprint document for the undergraduate program in software engineering and computing technology to support test developers and content specialists in creating valid and reliable assessments. The program has four streams (Artificial Intelligence, Cyber Security, Information Technology, and Software Engineering). The key considerations in the process of preparing this test blueprint guideline were the core competencies that have been already identified for the themes of courses, the course contents, course credit hours, the program streams, and the learning outcomes with their corresponding levels of achievement by learning domains.

In line with these, the number of test items that should adequately assess the performance of students in all the content topics will be determined through discussion with the content specialists and content developers.

2. The Objective of the Test Blueprint

The preparation of the test blueprint is aimed at facilitating the creation of a comprehensive assessment that accurately represents and samples the entire knowledge domain expected of software engineering and computing technology graduates.

The specific objectives of the test blueprint are to:

- Facilitate the construction of representative and balanced test items for the selected courses by the competencies identified.
- Guide test developers or writers to write or set appropriate test items.

3. Steps for the Development of the Test Blueprint

- Listing the identified core competencies corresponding to the respective course
- Listing the specific and minimum competencies to be assessed by the exam
- Listing out the major learned course contents corresponding to these listed minimum competencies.
- Listing the specific, measurable learning outcomes under these minimum competencies using action verbs (Note: the number and the importance of these learning outcomes proportionally contribute to the total test items. The weight of the categories of the list of learning outcomes under each competency and content should lead to the number of total test items to be constructed)
- Determination of the test format
- Content weighting - Experts judge the weight to be given to the listed learning outcomes and their corresponding contents.
- Identifying the categories of learning domains to which each of these learning outcomes belongs using their action verbs.
- Construction of test blueprint table
- Allocating the number of test items for each learning outcome by their corresponding types and levels of learning domains.
- Determination of the sub-total and total test items by their categories of learning domains and learning outcomes.

4. Components Considered in the Test Blueprint Preparation

Several core components of the curriculum are considered during test blueprint preparation to ensure the validity, reliability, and fairness of the assessment. (See Appendix A of Curricula of BSc in Software Engineering and Computing Technology of Addis Ababa Institute of Technology, Addis Ababa University)

4.1. Program Structure

The undergraduate program in **Software Engineering and Computing Technology** follows a structured framework comprising *three hierarchical tiers of academic semesters*.

Firstly, there is a pre-engineering semester aimed at acquainting students with various engineering fields, their roles, and preliminary courses. This phase encompasses language, writing skills, and social sciences/humanities courses. Its primary objective is to solidify students' educational foundations for higher learning and facilitate their adaptation to the academic system.

The second phase consists of five semesters where students engage with **Core Courses** in Software Engineering and Computing Technology. These courses cover areas such as *Computer Science, Mathematical Computer Science, Information Systems, Software Engineering, and Electrical Engineering*.

In the final two academic years, students select one of the specialization streams: **Software Engineering, Information Technology, Artificial Intelligence, or Cybersecurity**. Throughout this period, students focus on their chosen specialization and undertake courses tailored to their respective streams.

4.2. Graduate Profiles

This exit exam is expected to measure the software engineering and computing technology graduates' knowledge, skills, and attitudes. So software engineering and computing technology graduates are expected to demonstrate the following knowledge, skills, and attitudes:

Knowledge

- Have an understanding of professional, ethical, legal, security, and social issues and responsibilities.
- Have the ability to apply knowledge of computing and mathematics to resolve on-hand and upcoming problems.
- Have recognition of the need for, and an ability to engage in, continuing professional development and the knowledge and skills to act as research assistants or lecturers in higher education institutions
- Use current techniques, skills, and tools necessary for software development, testing and

maintenance.

- Have the knowledge and innovative skills to plan, set up and run computer related businesses.
- Have an understanding of best practices and standards and their application;

Skill

- The graduates will be able to demonstrate an understanding of and apply appropriate theories, models, and techniques that provide a basis for problem identification and analysis, design, development, implementation, verification, and documentation of computer systems.

Attitude

- Function effectively on teams to accomplish a common goal and work under stressful situations.
- Communicate effectively with a range of audiences and give clear, good-quality verbal presentations on scientific topics.
- Demonstrate an understanding and appreciation of the importance of negotiation, effective work habits, leadership, and good communication with stakeholders in a typical software development environment.
- Learn new models, techniques, and technologies as they emerge and appreciate the necessity of such continuing professional development.

4.3. Objectives of the Program

General Objective

The general objective of the Software Engineering and Computing Technology program is to provide a quality software engineering and computing education with significant hands-on and laboratory experience that will enable graduates to practice their profession with proficiency and integrity.

- Teach students to build upon and adapt knowledge of science, mathematics, engineering, and new computing technologies to take on more expansive tasks that require an increased level of self-reliance, technical expertise, and leadership.
- Equip students with the necessary methodologies, skills, and tools to develop and deploy high standard computer systems.
- Teach students to design, develop, test, document and maintain computer systems of an industrial standard for private, non-governmental and governmental clients.
- Train students to conduct research in computing domains such as High-Performance Computing, Natural Language Processing, Computer Vision, Dialogue Systems, OCR, Learning Systems, digital forensics, Security policy and risk analysis principles, and technologies emerging every day for socio-economic, and social problems and etc.

- Train students with entrepreneurship mindset in technology so that they can create new jobs, or expand existing business.
- Engage students in the global online competition so that their understanding meets the global scale.
- To train students to be effective team members, aware of cultural diversity, and who conduct themselves ethically and professionally.
- To enhance students' communication and technical skills to ensure the production of quality computer systems on time and within budget.

Specific Objectives

The following list of salient activities will be undertaken to achieve the general objective of the program:

- Conduct investigation of complex problems using research-based knowledge and research methods, including design of experiments, analysis, and interpretation of data, and synthesis of information to provide valid conclusions.
- Apply knowledge of mathematics and software engineering specialization to address complex and large-scale problems.
- Design integrated and efficient software solutions, components, or processes to address complex problems and implement them to meet the specified needs with appropriate consideration for public health and safety, cultural, societal (local and international), and environmental considerations.
- Explore fundamental concepts of object-oriented programming, including principles, object technology, and the programming environment, with an emphasis on key features like inheritance, encapsulation, polymorphism, exception handling, and I/O functionality.
- Gain hands-on experience in creating event-driven GUI containers and components within an integrated development environment.
- Embrace and implement cutting-edge software design and development tools, methodologies, and emerging technologies while understanding their potential capabilities, limitations, and associated risks.
- Demonstrate a commitment to ethical principles, and uphold professional ethics, responsibilities, and norms within the realm of software engineering practice.
- Adopt, innovate, select, and apply appropriate state-of-the-art software design and development tools, methodologies, resources, and emerging technologies.
- Apply ethical principles and commit to professional ethics, responsibilities, and norms of software engineering practice.
- Apply logical reasoning and informed decision-making to assess societal health, safety, privacy,

security, legal and cultural issues, and the consequent responsibilities relevant to software engineering application.

- Understand the impact of software applications in societal and environmental contexts and apply the principle of green computing for sustainable development.
- Apply ethical principles and commit to professional ethics and responsibilities and norms of software engineering practice.
- Identify, formulate, research literature, and analyze complex problems, reaching substantiated (justifiable, validated) conclusions using software development principles, methodologies, and tools.
- Demonstrate knowledge and skill on software project management principles and apply these to manage software development processes and its resource requirements, costs in multidisciplinary projects.
- Function effectively as an independent software engineer, as a member, or as a leader in a diverse team and multidisciplinary settings.
- Gain a comprehensive understanding of the fundamental concepts that form the backbone of computer networks.
- Analyze the different networking layers and design appropriate computer network services based on a given requirement.
- Apply ethical principles and commit to professional ethics and responsibilities and norms of software engineering practice.

4.4. Learning Outcomes

- Identify the basic concepts of programming with its program development process
- Apply the basic programming constructors' and the various programming techniques
- Demonstrate problem-solving skills by analyzing a variety of real-world problems using problem-solving techniques
- Analyze modular programming, user-defined data type; file stream handling and vectors by solving variety programming problems.
- Develop, debug, and test medium-scale applications by analyzing a variety of real-world problems
- Follow the ability of self-education and life-long learning needed by computer programming
- State how data is organized in a computer, how it can be retrieved, and how it can be used
- Explain the basic techniques for the design and analysis of efficient Algorithm · Apply various data structures including Linked lists, Queues, Stacks, Binary trees, Search trees and Graphs
- Distinguish complexity, efficiency of searching and sorting algorithms using Empirical and

Theoretical analysis

- Design algorithms to solve real-life problems
- Discuss major concepts of object-oriented programming, object technology and programming environment.
- Use inheritance, encapsulation, polymorphism exception handling and I/O functionality
- Create event-driven GUI containers and components within an integrated development environment
- Design and develop object-oriented software
- Recognize a web protocol, design and development of web-based application · Use modern tools and techniques for developing web-based applications.
- Develop websites, web frameworks using basic HTML, CSS, JavaScript, AJAX and Server-Side scripting
- Create dynamic and interactive web pages as well as the ability to connect to a wide range of databases
- Assess web-based solution and engage in independent and lifelong learning in the broad scope of Internet programming as well as web application · State the basic components of an Android application and mobile computing · Demonstrate basic skills mobile computing of using an integrated development environment
- Evaluate the security concerns of the mobile computing technologies and mobile networks.
- Choose the legal and ethical principles of the mobile application framework for designing and constructing mobile Software applications
- Explain fundamentals of database system concepts, technology, models of database and optimization.
- Write SQL statements for data creation and manipulation purposes
- Design the database models from specifications using a popular DBMS
- Manage the possible problems that may occur in securing data.
- Recall the structure and functionality of modern Operating Systems
- Explain conditions that lead to deadlock and memory management, I/O, mass storage and file management issues
- Describe security and protection measures used in operating systems · Use knowledge of process management and synchronization for software development
- Compare the various algorithms on which the core functions of the Operating Systems are built on
- Identifies the basic principles of Software Engineering
- Describe software process and state-of-the-arts software process models
- Explain the fundamental concept of data communication, computer networks, and computer system security as well as setup and configure small-area network.
- Demonstrate installation and configuration of system software and make use of software engineering tools.

- Be aware of the professional, ethical, and norms of software engineering practice.
- Describe concepts of network fundamentals.
- Analyze the different networking layers
- Design appropriate computer network services based on a given requirement
- Identify malicious code functions, threats, vulnerabilities, possible propagation, and mitigation methods
- Assess computing cryptography and cryptanalysis
- Develop mechanisms for authentication and access control in a domain.
- Compare types of firewall and intrusion detection methods
- Explain fundamental concepts of artificial intelligent agents
- Compare the basic search strategies used in an intelligent agent.
- Apply the law of logic to represent knowledge for an expert system
- Engage in independent and lifelong learning with the broad application area of Artificial Intelligence.
- Explain the distinction between supervised and unsupervised learning, as well as the interests and difficulties of both approaches.
- Implement the major machine learning algorithms for data analytics
- Formulate research on a personal project using machine learning techniques
- Apply modern software testing processes about software development and project management
- Create test strategies and plans, design test cases, prioritize and execute them
- Manage incidents and risks within a project
- Become a competent expert in designing, implementation and development of Quality software systems and IT processes
- Translate problems into search algorithms & find ideal solutions.
- Analyze, formalize, select search methods, and implement A* & iterative deepening.
- Solve complex problems with backtracking & local search techniques.
- Implement alpha-beta search, and design winning strategies & evaluations.
- Proof-based solutions & classical planning approach.
- Formulate machine learning problems corresponding to different applications.
- Understand a range of machine learning algorithms along with their strengths and weaknesses.
- Understand the basic theory underlying machine learning.
- Be able to apply machine learning algorithms to solve problems of moderate complexity.
- Apply machine learning algorithms for classification and problems in health-related domains like medicine, genetics, and medical diagnosis.

5. Themes and List of Courses

To help demarcate the focus areas and competency boundaries covered by the identified courses, the courses have been categorized under five themes. Accordingly, **Table 1** maps courses to the respective themes identified.

Table 1: Courses categorization based on themes and streams

Theme	Courses	Cr.Hrs
Problem Analysis and Programming (60.38%)	1.Fundamentals of Programming	8
	2.Fundamentals of Data Structures and Algorithms	4
	3.Object Oriented Programming	3
	4.Internet Programming (Web Programming)	6
	5.Mobile Application Development	3
	6.Fundamentals of Databases	4
	7.Operating Systems	4
Software requirement, Design, and Architecture (15.09%)	8.Fundamentals of Software Engineering	4
	9.Software Project Management	4
Networking and Software Security (13.21%)	10.Fundamentals of Networking	4
	11.Software and Information Security	3
Emerging Technologies (11.32%)	12.Fundamentals of AI	3
	13.Machine Learning	3
Total		53

Share of the Themes/Courses/Items in Percentage (%) (Please see Table 2

○ Share of themes (T) = $\frac{a}{b} * 100$

Where “a” is the credit hour of a theme and “b” is the total credit hour of the program. The credit hour of a theme is the sum of credit hours of courses in the theme.

○ Share of courses per theme (C) = $\frac{\text{Credit hour of the course}}{\text{Credit hour of the theme}} * 100$

○ Share of items per course = $\frac{\text{Share of the course}}{\text{Total number of items}}$

- Learning outcomes in terms of the three domains (Cognitive, Affective and Psychomotor)
- Test Blueprint Table (Table of Specification) (See Table 2 below)

5.1. Common Core Courses Theme, Courses, and Learning Outcomes

Theme	General Objective	Course	Credit Hour	Specific Objectives/ Learning Outcomes	No Item per Learning Outcome	Learning Outcomes							No of Items		
						Cognitive								Motor	Affective
						Remembering	Understanding	Application	Analysis	Evaluation	Creation/Synthesis				
Problem Analysis and Programming (60.38%)	- Conduct investigation of complex problems using research-based knowledge and research methods, including design of experiments, analysis, and interpretation of data, and synthesis of information to provide valid conclusions. - Apply knowledge of mathematics and software engineering specialization to address complex and large-scale problems.	Fundamentals of Programming	8	Identify the basic concepts of programming with its program development process Apply the basic programming constructors' and the various programming techniques Demonstrate problem-solving skills by analyzing a variety of real-world problems using program-solving techniques Analyze modular programming, user-defined data type; file stream handling and vectors by solving variety programming problems Develop, debug, and test medium-scale applications by analyzing a variety of real-world problems	1	1								14	
					3	3									
					3			1	2						
					4	2	2								
					3	1		1	1						
					2	1	1								
					2	1	1								
					1										
					1	1									
					2										
					2										
					2										
					2										
					2										
	- Conduct investigation of complex problems using research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions. - Design integrated and efficient software solutions, components, or processes to address complex problems and implement them to meet the specified needs with appropriate consideration for public health and safety, cultural, societal (local and international), and environmental considerations.	Fundamentals of Data Structures and Algorithms	4	State how data is organized in a computer, how it can be retrieved, and how it can be used Explain the basic techniques for the design and analysis of efficient Algorithm Apply various data structures including Linked lists, Queues, Stacks, Binary trees, Search trees, and Graphs Distinguish complexity and efficiency of searching and sorting algorithms using Empirical and Theoretical analysis Design algorithms to solve real-life problems	2	1	1						8		
					2	1	1								
					1										
					1	1									
					1										
					1										
					1										
					1										
					1										
					1										
					1										
					1										
					1										
					1										
1															

Emerging Technologies (11.32%)	backbone of computer networks. • Analyze the different networking layers and design appropriate computer network services based on a given requirement.				3	1	1			1			
	Software and Information Security	3	Identify malicious code functions, threats, vulnerabilities, possible propagation, and mitigation methods	Assess computing cryptography and cryptanalysis	1		1						
				Develop mechanisms for authentication and access control in a domain	2	1	1						
				Compare types of firewall and intrusion detection methods	1		1						
	Fundamentals of AI	3	Gain a comprehensive understanding of AI principles, including problem-solving, reasoning, planning, and machine learning. • Develop proficiency in applying AI techniques to solve real-world problems autonomously. • Explore the interdisciplinary connections between AI and related fields	Recall and understand AI search problem formulations and key concepts	2	1				1			
				Apply search methods and algorithms to solve diverse problems effectively	2		1			1			
				Analyze complexities of search algorithms and conditional independence in Bayes Nets	2	1		1					
	Machine Learning	3	Gain understanding of machine learning principles and their implementation processes. • Develop proficiency in designing and implementing various machine learning algorithms. • Enhance problem-solving skills by tackling practical machine learning applications.	Understand ML understanding algorithmic theories comprehensively and theory of machine learning	2		1			1			
				Formulating ML problems	1					1			
				Applying ML algorithms effectively across diverse domains for practical solutions.	2		1			1			
				Developing ML-based applications with proficiency and practical problem-solving skills.	1							1	
6													
6													

6. Conclusion

Exit examinations play an indispensable role in fostering erudite, proficient, and cognitively developed graduates, serving as a litmus test for program efficacy. These assessments serve as pivotal checkpoints, ensuring that graduates possess the requisite knowledge, skills, and attitudes deemed essential for success in their respective fields. By subjecting students to rigorous evaluations encompassing a breadth of subject matter and cognitive domains, exit examinations provide a comprehensive measure of academic attainment, enabling educators to gauge the effectiveness of their instructional methodologies and curriculum design. Moreover, these examinations catalyze continuous improvement, driving educators to refine and enhance academic programs to better meet the evolving needs and expectations of the workforce and society at large.

In addition to their evaluative function, exit examinations contribute substantively to enhancing the caliber and efficacy of academic programs. By providing a standardized benchmark for performance evaluation, these assessments facilitate benchmarking and comparative analysis across institutions, fostering healthy competition and driving excellence in educational outcomes. Furthermore, exit examinations serve as a catalyst for synergistic collaboration among diverse universities, fostering knowledge sharing and best practice dissemination. Through collaborative efforts to refine examination standards and align curricular objectives, academic institutions can collectively raise the bar for educational excellence, ensuring that graduates are well-equipped to navigate the complexities of their chosen professions and contribute meaningfully to society.

Appendix A:

See Curricula of B.Sc. in Software Engineering and Computing Technologies

Draft